

DATA STRUCTURES

Year : 2023-2024
Semester : **B.Tech III Semester**
Branch : CSE

INTRODUCTION TO DATA STRUCTURES,SEARCHING AND SORTING

Basic Concepts: Introduction to Data Structures:

A data structure is a way of storing data in a computer so that it can be used efficiently and it will allow the most efficient algorithm to be used. The choice of the data structure begins from the choice of an abstract data type (ADT). A well-designed data structure allows a variety of critical operations to be performed, using as few resources, both execution time and memory space, as possible. Data structure introduction refers to a scheme for organizing data, or in other words it is an arrangement of data in computer's memory in such a way that it could make the data quickly available to the processor for required calculations.

A data structure should be seen as a logical concept that must address two fundamental concerns.

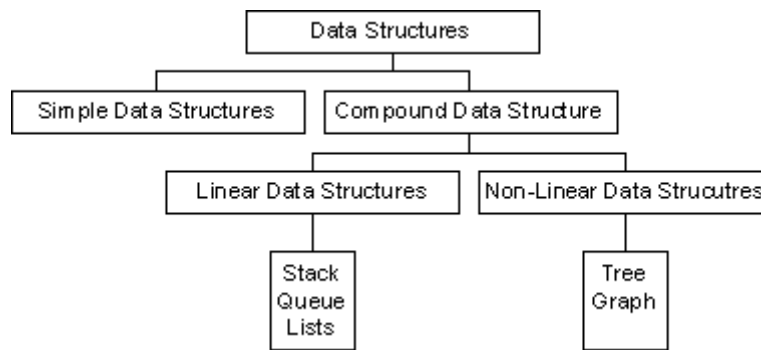
1. First, how the data will be stored, and
2. Second, what operations will be performed on it.

As data structure is a scheme for data organization so the functional definition of a data structure should be independent of its implementation. The functional definition of a data structure is known as ADT (Abstract Data Type) which is independent of implementation. The way in which the data is organized affects the performance of a program for different tasks. Computer programmers decide which data structures to use based on the nature of the data and the processes that need to be performed on that data. Some of the more commonly used data structures include lists, arrays, stacks, queues, heaps, trees, and graphs.

Classification of Data Structures:

Data structures can be classified as

- Simple data structure
- Compound data structure
- Linear data structure
- Non linear data structure



[Fig 1.1 Classification of Data Structures]

Simple Data Structure:

Simple data structure can be constructed with the help of primitive data structure. A primitive data structure used to represent the standard data types of any one of the computer languages. Variables, arrays, pointers, structures, unions, etc. are examples of primitive data structures.

Compound Data structure:

Compound data structure can be constructed with the help of any one of the primitive data structure and it is having a specific functionality. It can be designed by user. It can be classified as

- Linear data structure
- Non-linear data structure

Linear Data Structure:

Linear data structures can be constructed as a continuous arrangement of data elements in the memory. It can be constructed by using array data type. In the linear Data Structures the relationship of adjacency is maintained between the data elements.

Operations applied on linear data structure:

The following list of operations applied on linear data structures

1. Add an element
2. Delete an element
3. Traverse
4. Sort the list of elements
5. Search for a data element

For example Stack, Queue, Tables, List, and Linked Lists.

Non-linear Data Structure:

Non-linear data structure can be constructed as a collection of randomly distributed set of data item joined together by using a special pointer (tag). In non-linear Data structure the relationship of adjacency is not maintained between the data items.

Operations applied on non-linear data structures:

The following list of operations applied on non-linear data structures.

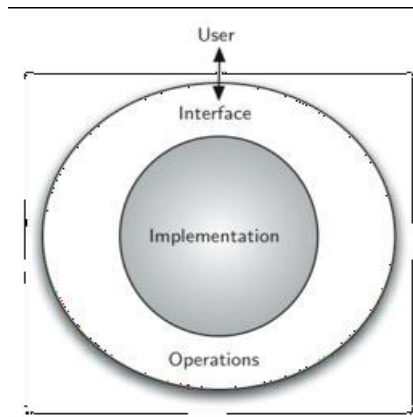
1. Add elements
2. Delete elements
3. Display the elements
4. Sort the list of elements

5. Search for a data element

For example Tree, Decision tree, Graph and Forest

Abstract Data Type:

An abstract data type, sometimes abbreviated ADT, is a logical description of how we view the data and the operations that are allowed without regard to how they will be implemented. This means that we are concerned only with what data is representing and not with how it will eventually be constructed. By providing this level of abstraction, we are creating an encapsulation around the data. The idea is that by encapsulating the details of the implementation, we are hiding them from the user's view. This is called information hiding. The implementation of an abstract data type, often referred to as a data structure, will require that we provide a physical view of the data using some collection of programming constructs and primitive data types.



[Fig. 1.2: Abstract Data Type (ADT)]

Algorithms:

Structure and Properties of Algorithm:

An algorithm has the following structure

1. Input Step
2. Assignment Step
3. Decision Step
4. Repetitive Step
5. Output Step

An algorithm is endowed with the following properties:

1. **Finiteness:** An algorithm must terminate after a finite number of steps.
2. **Definiteness:** The steps of the algorithm must be precisely defined or unambiguously specified.
3. **Generality:** An algorithm must be generic enough to solve all problems of a particular class.
4. **Effectiveness:** the operations of the algorithm must be basic enough to be put down on pencil and paper. They should not be too complex to warrant writing another algorithm for the operation.
5. **Input-Output:** The algorithm must have certain initial and precise inputs, and outputs that may be generated both at its intermediate and final steps.

Different Approaches to Design an Algorithm:

An algorithm does not enforce a language or mode for its expression but only demands adherence to its properties.

Practical Algorithm Design Issues:

1. **To save time (Time Complexity):** A program that runs faster is a better program.
2. **To save space (Space Complexity):** A program that saves space over a competing program is

considerable desirable.

Efficiency of Algorithms:

The performances of algorithms can be measured on the scales of **time** and **space**. The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical and the other is experimental. In performance analysis we use analytical methods, while in performance measurement we conduct experiments.

Time Complexity: The time complexity of an algorithm or a program is a function of the running time of the algorithm or a program. In other words, it is the amount of computer time it needs to run to completion.

Space Complexity: The space complexity of an algorithm or program is a function of the space needed by the algorithm or program to run to completion.

The time complexity of an algorithm can be computed either by an **empirical** or **theoretical** approach. The **empirical** or **posteriori testing** approach calls for implementing the complete algorithms and executing them on a computer for various instances of the problem. The time taken by the execution of the programs for various instances of the problem are noted and compared. The algorithm whose implementation yields the least time is considered as the best among the candidate algorithmic solutions.

Analyzing Algorithms

Suppose M is an algorithm, and suppose n is the size of the input data. Clearly the complexity $f(n)$ of M increases as n increases. It is usually the rate of increase of $f(n)$ with some standard functions. The most common computing times are

$O(1)$, $O(\log_2 n)$, $O(n)$, $O(n \log_2 n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$

Example:

Program Segment A

```

-----
x = x + 2;
-----

```

Program Segment B

```

-----
for k = 1 to n do
    x = x + 2;
end;
-----

```

Program Segment C

```

-----
for j = 1 to n do
    for x = 1 to n do
        x = x + 2;
    end
end;
-----

```

Total Frequency Count of Program Segment A

Program Statements	Frequency Count
----- x = x + 2; -----	1
Total Frequency Count	1

Total Frequency Count of Program Segment B

Program Statements	Frequency Count
----- for k = 1 to n do x = x + 2; end; -----	(n+1) n n
Total Frequency Count	3n+1

Total Frequency Count of Program Segment C

Program Statements	Frequency Count
----- for j = 1 to n do for x = 1 to n do x = x + 2; end end; -----	(n+1) n(n+1) n ² n ² n
Total Frequency Count	3n ² +3n+1

The total frequency counts of the program segments A, B and C given by 1, (3n+1) and (3n²+3n+1) respectively are expressed as O(1), O(n) and O(n²). These are referred to as the time complexities of the program segments since they are indicative of the running times of the program segments. In a similar manner space complexities of a program can also be expressed in terms of mathematical notations,

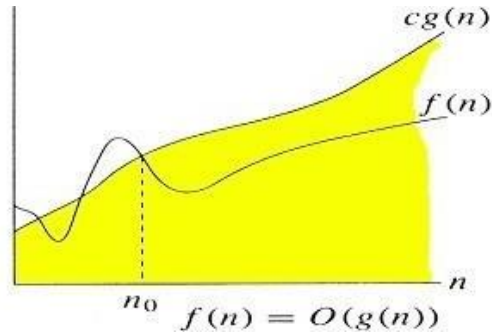
which is nothing but the amount of memory they require for their execution.

Asymptotic Notations:

It is often used to describe how the size of the input data affects an algorithm's usage of computational resources. Running time of an algorithm is described as a function of input size n for large n .

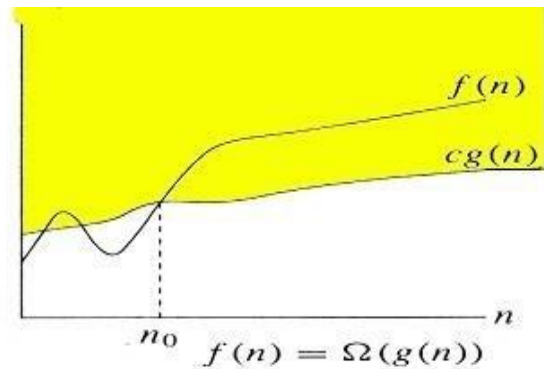
Big oh(O): Definition: $f(n) = O(g(n))$ (read as f of n is big oh of g of n) if there exist a positive integer n_0 and a positive number c such that $|f(n)| \leq c|g(n)|$ for all $n \geq n_0$. Here $g(n)$ is the upper bound of the function $f(n)$.

$f(n)$	$g(n)$	
$16n^3 + 45n^2 + 12n$	n^3	$f(n) = O(n^3)$
$34n - 40$	n	$f(n) = O(n)$
50	1	$f(n) = O(1)$



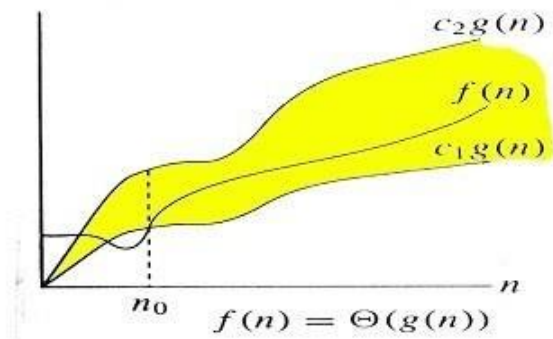
Omega(Ω): Definition: $f(n) = \Omega(g(n))$ (read as f of n is omega of g of n), if there exists a positive integer n_0 and a positive number c such that $|f(n)| \geq c|g(n)|$ for all $n \geq n_0$. Here $g(n)$ is the lower bound of the function $f(n)$.

$f(n)$	$g(n)$	
$16n^3 + 8n^2 + 2$	n^3	$f(n) = \Omega(n^3)$
$24n + 9$	n	$f(n) = \Omega(n)$



Theta(Θ): Definition: $f(n) = \Theta(g(n))$ (read as f of n is theta of g of n), if there exists a positive integer n_0 and two positive constants c_1 and c_2 such that $c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$ for all $n \geq n_0$. The function $g(n)$ is both an upper bound and a lower bound for the function $f(n)$ for all values of n , $n \geq n_0$.

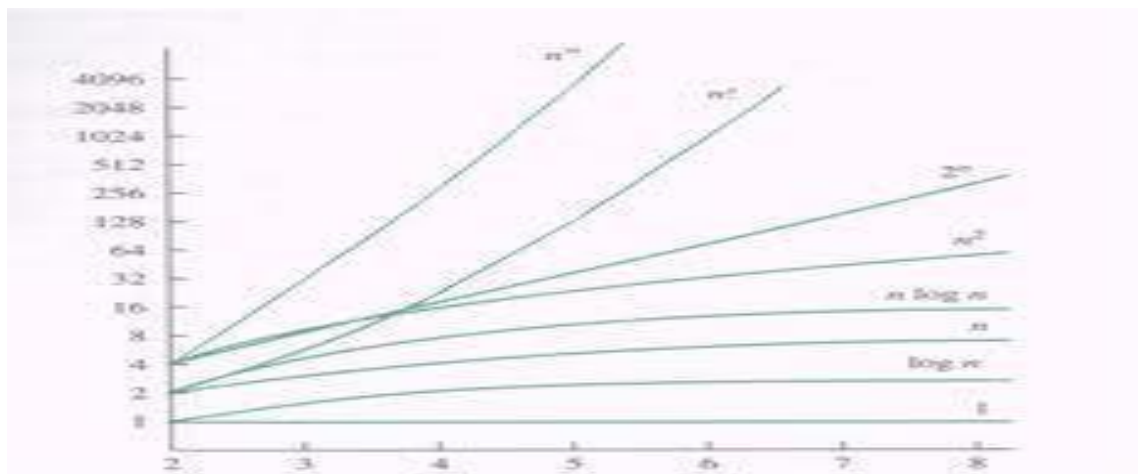
$f(n)$	$g(n)$	
$16n^3 + 30n^2 - 90$	n^2	$f(n) = \Theta(n^2)$
$7 \cdot 2^n + 30n$	2^n	$f(n) = \Theta(2^n)$



Little oh(o): Definition: $f(n) = O(g(n))$ (read as f of n is little oh of g of n), if $f(n) = O(g(n))$ and $f(n) \neq \Omega(g(n))$.

Time Complexity:

Time Complexities of various Algorithms:



Numerical Comparision of Different Algorithms:

S.No.	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n
1.	0	1	1	1	1	2
2.	1	2	2	4	8	4
3.	2	4	8	16	64	16
4.	3	8	24	64	512	256
5.	4	16	64	256	4096	65536

Reasons for analyzing algorithms:

1. To predict the resources that the algorithm requires

- Computational Time(CPU consumption).
- Memory Space(RAM consumption).
- Communication bandwidth consumption.

2. To predict the running time of an algorithm

- Total number of primitive operations executed.

Recursive Algorithms:

GCD Design: Given two integers a and b, the greatest common divisor is recursively found using the formula

$$\text{gcd}(a,b) = \begin{cases} a & \text{if } b=0 \\ b & \text{if } a=0 \\ \text{gcd}(b, a \bmod b) & \end{cases}$$

Fibonacci Design: To start a fibonacci series, we need to know the first two numbers.

$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{Fibonacci}(n-1) + \text{fibonacci}(n-2) & \end{cases}$$

Difference between Recursion and Iteration:

1. A function is said to be recursive if it calls itself again and again within its body whereas iterative functions are loop based imperative functions.
2. Recursion uses stack whereas iteration does not use stack.
3. Recursion uses more memory than iteration as its concept is based on stacks.
4. Recursion is comparatively slower than iteration due to overhead condition of maintaining stacks.
5. Recursion makes code smaller and iteration makes code longer.
6. Iteration terminates when the loop-continuation condition fails whereas recursion terminates when a base case is recognized.
7. While using recursion multiple activation records are created on stack for each call whereas in iteration everything is done in one activation record.
8. Infinite recursion can crash the system whereas infinite looping uses CPU cycles repeatedly.
9. Recursion uses selection structure whereas iteration uses repetition structure.

Types of Recursion:

Recursion is of two types depending on whether a function calls itself from within itself or whether two functions call one another mutually. The former is called **direct recursion** and the latter is called

indirect recursion. Thus there are two types of recursion:

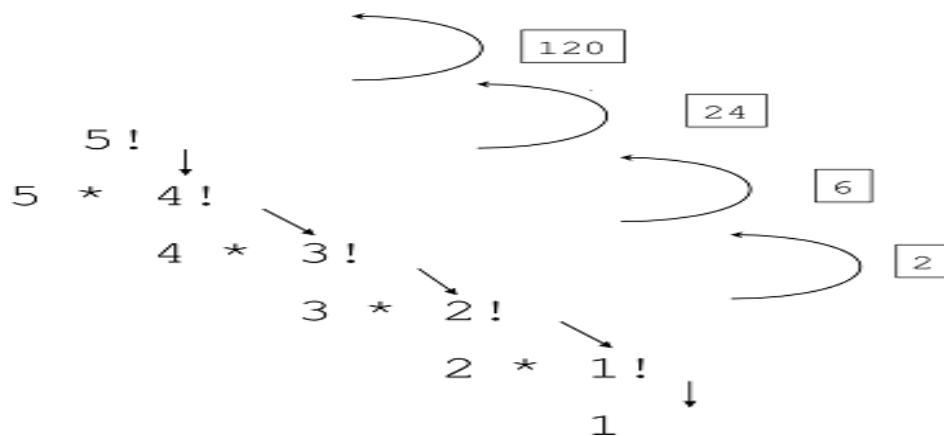
- Direct Recursion
- Indirect Recursion

Recursion may be further categorized as:

- Linear Recursion
- Binary Recursion
- Multiple Recursion

Linear Recursion:

It is the most common type of Recursion in which function calls itself repeatedly until base condition [termination case] is reached. Once the base case is reached the results are return to the caller function. If a recursive function is called only once then it is called a linear recursion.



Binary Recursion:

Some recursive functions don't just have one call to themselves; they have two (or more). Functions with two recursive calls are referred to as binary recursive functions.

Example1: The Fibonacci function fib provides a classic example of binary recursion. The Fibonacci numbers can be defined by the rule:

$$\begin{aligned} \text{fib}(n) &= 0 \text{ if } n \text{ is } 0, \\ &= 1 \text{ if } n \text{ is } 1, \\ &= \text{fib}(n-1) + \text{fib}(n-2) \text{ otherwise} \end{aligned}$$

For example, the first seven Fibonacci numbers are

$$\text{Fib}(0) = 0$$

$$\text{Fib}(1) = 1$$

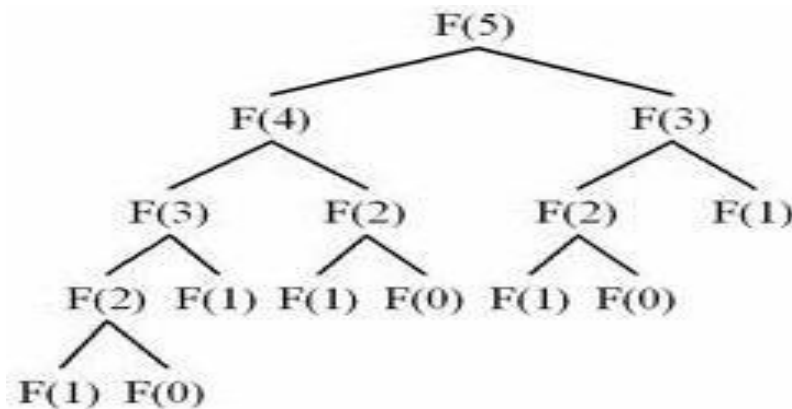
$$\text{Fib}(2) = \text{Fib}(1) + \text{Fib}(0) = 1$$

$$\text{Fib}(3) = \text{Fib}(2) + \text{Fib}(1) = 2$$

$$\text{Fib}(4) = \text{Fib}(3) + \text{Fib}(2) = 3$$

$$\text{Fib}(5) = \text{Fib}(4) + \text{Fib}(3) = 5$$

$$\text{Fib}(6) = \text{Fib}(5) + \text{Fib}(4) = 8$$



Program to display the Fibonacci sequence up to n-th term where n is provided by the user

change this value for a different result

nterms = 10

uncomment to take input from the user

#nterms = int(input("How many terms? "))

first two terms

n1 = 0

n2 = 1

count = 0

check if the number of terms is valid

if nterms <= 0:

 print("Please enter a positive integer")

elif nterms == 1:

 print("Fibonacci sequence upto",nterms,":")

 print(n1)

else:

```

print("Fibonacci sequence upto",nterms,":")
while count < nterms:
    print(n1,end=' , ')
    nth = n1 + n2
    # update values
    n1 = n2
    n2 = nth
    count += 1

```

Tail Recursion:

Tail recursion is a form of linear recursion. In tail recursion, the recursive call is the last thing the function does. Often, the value of the recursive call is returned. As such, tail recursive functions can often be easily implemented in an iterative manner; by taking out the recursive call and replacing it with a loop, the same effect can generally be achieved. In fact, a good compiler can recognize tail recursion and convert it to iteration in order to optimize the performance of the code.

A good example of a tail recursive function is a function to compute the GCD, or Greatest Common Denominator, of two numbers:

```

def factorial(n):
    if n == 0: return 1
    else: return factorial(n-1) * n

def tail_factorial(n, accumulator=1):
    if n == 0: return 1
    else: return tail_factorial(n-1, accumulator * n)

```

Recursive algorithms for Factorial, GCD, Fibonacci Series and Towers of Hanoi:Factorial(n)

Input: integer $n \geq 0$

Output: $n!$

1. If $n = 0$ then return (1)
2. else return $\text{prod}(n, \text{factorial}(n - 1))$

GCD(m, n)

Input: integers $m > 0, n \geq 0$

Output: $\text{gcd}(m, n)$

1. If $n = 0$ then return (m)
2. else return $\text{gcd}(n, m \bmod n)$

Time-Complexity: $O(\ln n)$

Fibonacci(n)

Input: integer $n \geq 0$

Output: Fibonacci Series: 1 1 2 3 5 8 13.....

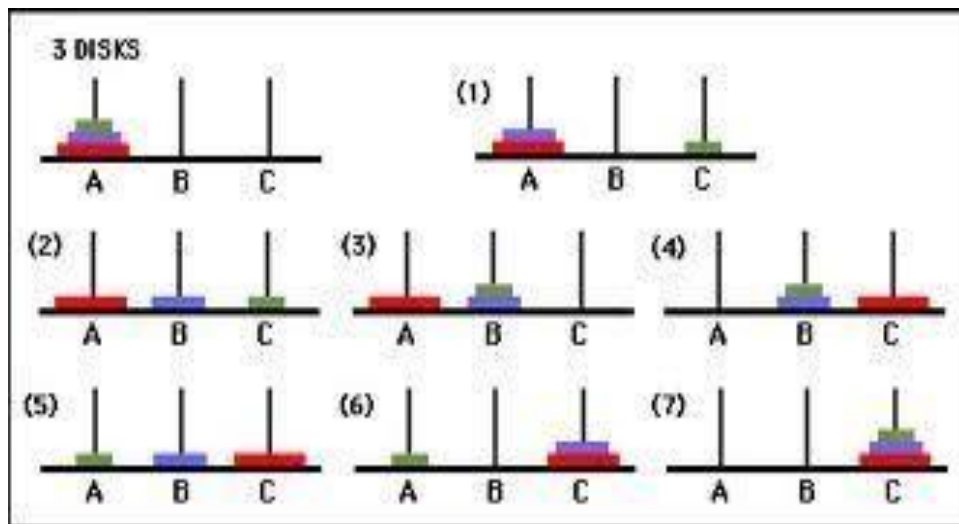
1. if $n=1$ or $n=2$
2. then $\text{Fibonacci}(n)=1$
3. else $\text{Fibonacci}(n) = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$

Towers of Hanoi

Input: The aim of the tower of Hanoi problem is to move the initial n different sized disks from needle A to needle C using a temporary needle B. The rule is that no larger disk is to be placed above the smaller disk in any of the needle while moving or at any time, and only the top of the disk is to be moved at a time from any needle to any needle.

Output:

1. If $n=1$, move the single disk from A to C and return,
2. If $n>1$, move the top $n-1$ disks from A to B using C as temporary.
3. Move the remaining disk from A to C.
4. Move the $n-1$ disk disks from B to C, using A as temporary.



```
def TowerOfHanoi(n , from_rod, to_rod, aux_rod):
```

```

if n == 1:

    print "Move disk 1 from rod",from_rod,"to rod",to_rod

    return

TowerOfHanoi(n-1, from_rod, aux_rod, to_rod)

print "Move disk",n,"from rod",from_rod,"to rod",to_rod

TowerOfHanoi(n-1, aux_rod, to_rod, from_rod)

n = 4

TowerOfHanoi(n, 'A', 'C', 'B')

```

Searching Techniques:

Linear Search: Searching is a process of finding a particular data item from a collection of data items based on specific criteria. Every day we perform web searches to locate data items containing in various pages. A search typically performed using a **search key** and it answers either True or False based on the item is present or not in the list. Linear search algorithm is the most simplest algorithm to do sequential search and this technique iterates over the sequence and checks one item at a time, until the desired item is found or all items have been examined. In Python the **in** operator is used to find the desired item in a sequence of items. The **in** operator makes searching task simpler and hides the inner working details.

```

if key in theArray :
    print( "The key is in the array." )
else :
    print( "The key is not in the array." )

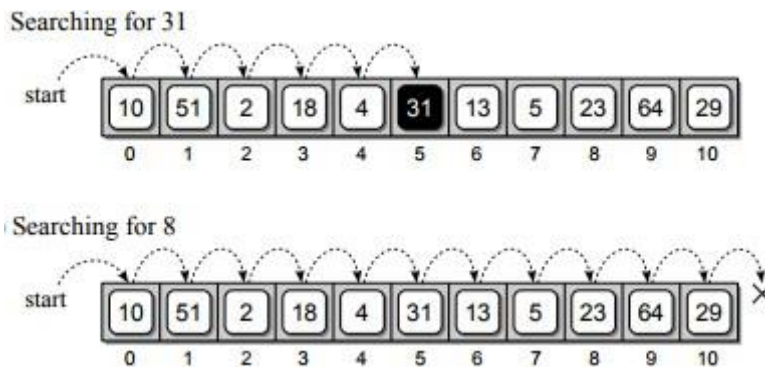
```

```

>>> 15 in [3,5,2,4,1]
False
>>> 3 in [3,5,2,4,1]
True
>>>

```

Consider an unsorted single dimensional array of integers and we need to check whether 31 is present in the array or not, then search begins with the first element. As the first element doesn't contain the desired value, then the next element is compared to value 31 and this process continues until the desired element is found in the sixth position. Similarly, if we want to search for 8 in the same array, then the search begins in the same manner, starting with the first element until the desired element is found. In linear search, we cannot determine that a given search value is present in the sequence or not until the entire array is traversed.



Source Code:

```
def linear_search(obj, item, start=0):
    for i in range(start, len(obj)):
        if obj[i] == item:
            return i
    return -1
arr=[1,2,3,4,5,6,7,8]
x=4
result=linear_search(arr,x)
if result==-1:
    print ("element does not exist")
else:
    print ("element exist in position %d" %result)
```

Time Complexity of Linear Search:

Any algorithm is analyzed based on the unit of computation it performs. For linear search, we need to count the number of comparisons performed, but each comparison may or may not search the desired item.

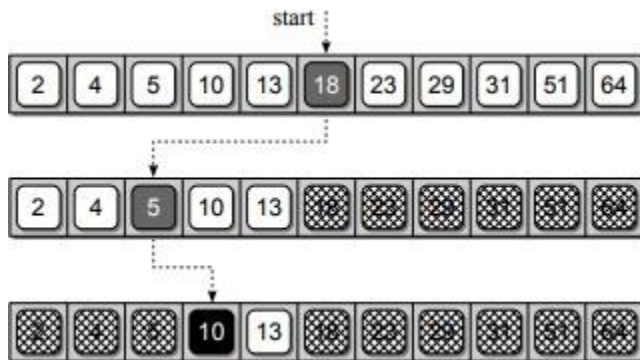
Case	Best Case	Worst Case	Average Case
If item is present	1	n	$n / 2$
If item is not present	n	n	n

Binary Search: In Binary search algorithm, the target key is examined in a sorted sequence and this algorithm starts searching with the middle item of the sorted sequence.

- If the middle item is the target value, then the search item is found and it returns True.
- If the target item < middle item, then search for the target value in the first half of the list.
- If the target item > middle item, then search for the target value in the second half of the list.

In binary search as the list is ordered, so we can eliminate half of the values in the list in each iteration. Consider an example, suppose we want to search 10 in a sorted array of elements, then we first determine

the middle element of the array. As the middle item contains 18, which is greater than the target value 10, so can discard the second half of the list and repeat the process to first half of the array. This process is repeated until the desired target item is located in the list. If the item is found then it returns True, otherwise False.



Searching for 10 in a sorted array using Binary Search

Code:

```
array =[1,2,3,4,5,6,7,8,9]

def binary_search(searchfor,array):
    lowerbound=0
    upperbound=len(array)-1
    found=False
    while found==False and lowerbound<=upperbound:
        midpoint=(lowerbound+upperbound)//2
        if array[midpoint]==searchfor:
            found =True
            return found
        elif array[midpoint]<searchfor:
            lowerbound=midpoint+1
        else:
            upperbound=midpoint-1
    return found

searchfor=int(input("what are you searching for?"))
if binary_search(searchfor,array):
    print ("element found")
else:
    print ("element not found")
```

Time Complexity of Binary Search:

In Binary Search, each comparison eliminates about half of the items from the list. Consider a list with n items, then about $n/2$ items will be eliminated after first comparison. After second comparison, $n/4$ items

of the list will be eliminated. If this process is repeated for several times, then there will be just one item left in the list. The number of comparisons required to reach to this point is $n/2^i = 1$. If we solve for i , then it gives us $i = \log n$. The maximum number of comparisons is logarithmic in nature, hence the time complexity of binary search is $O(\log n)$.

Case	Best Case	Worst Case	Average Case
If item is present	1	$O(\log n)$	$O(\log n)$
If item is not present	$O(\log n)$	$O(\log n)$	$O(\log n)$

Fibonacci Search: It is a comparison based technique that uses Fibonacci numbers to search an element in a sorted array. It follows divide and conquer approach and it has a $O(\log n)$ time complexity. Let the element to be searched is x , then the idea is to first find the smallest Fibonacci number that is greater than or equal to length of given array. Let the Fibonacci number be $\text{fib}(n^{\text{th}} \text{ Fibonacci number})$. Use $(n-2)^{\text{th}}$ Fibonacci number as index and say it is i , then compare $a[i]$ with x , if x is same then return i . Else if x is greater, then search the sub array after i , else search the sub array before i .

Source Code:

```
# Python3 program for Fibonacci search.
from bisect import bisect_left

# Returns index of x if present, else
# returns -1
def fibMonaccianSearch(arr, x, n):

    # Initialize fibonacci numbers
    fibMMm2 = 0 # (m-2)'th Fibonacci No.
    fibMMm1 = 1 # (m-1)'th Fibonacci No.
    fibM = fibMMm2 + fibMMm1 # m'th Fibonacci

    # fibM is going to store the smallest
    # Fibonacci Number greater than or equal to n
    while (fibM < n):
        fibMMm2 = fibMMm1
        fibMMm1 = fibM
        fibM = fibMMm2 + fibMMm1

    # Marks the eliminated range from front
    offset = -1;

    # while there are elements to be inspected.
    # Note that we compare arr[fibMm2] with x.
    # When fibM becomes 1, fibMm2 becomes 0
    while (fibM > 1):
```

```
# Check if fibMm2 is a valid location
i = min(offset+fibMMm2, n-1)
```

```
# If x is greater than the value at
# index fibMm2, cut the subarray array
# from offset to i
if (arr[i] < x):
    fibM = fibMMm1
    fibMMm1 = fibMMm2
    fibMMm2 = fibM - fibMMm1
    offset = i
```

```
# If x is greater than the value at
# index fibMm2, cut the subarray
# after i+1
elif (arr[i] > x):
    fibM = fibMMm2
    fibMMm1 = fibMMm1 - fibMMm2
    fibMMm2 = fibM - fibMMm1
```

```
# element found. return index
else :
    return i
```

```
# comparing the last element with x */
if(fibMMm1 and arr[offset+1] == x):
    return offset+1;
```

```
# element not found. return -1
return -1
```

```
# Driver Code
arr = [10, 22, 35, 40, 45, 50, 80, 82, 85, 90, 100]
n = len(arr)
x = 80
print("Found at index:",
      fibMonaccianSearch(arr, x, n))
```

Time Complexity of Fibonacci Search:

Time complexity for Fibonacci search is $O(\log_2 n)$

Sorting Techniques:

Sorting in general refers to various methods of arranging or ordering things based on criteria's (numerical, chronological, alphabetical, hierarchical etc.). There are many approaches to sorting data and each has its own merits and demerits.

Bubble Sort:

This sorting technique is also known as ***exchange sort***, which arranges values by iterating over the list several times and in each iteration the larger value gets bubble up to the end of the list. This algorithm uses multiple passes and in each pass the first and second data items are compared. if the first data item is bigger than the second, then the two items are swapped. Next the items in second and third position are compared and if the first one is larger than the second, then they are swapped, otherwise no change in their order. This process continues for each successive pair of data items until all items are sorted.

Bubble Sort Algorithm:

Step 1: Repeat Steps 2 and 3 for $i=1$ to 10

Step 2: Set $j=1$

Step 3: Repeat while $j \leq n$

(A) if $a[i] < a[j]$

Then interchange $a[i]$ and $a[j]$

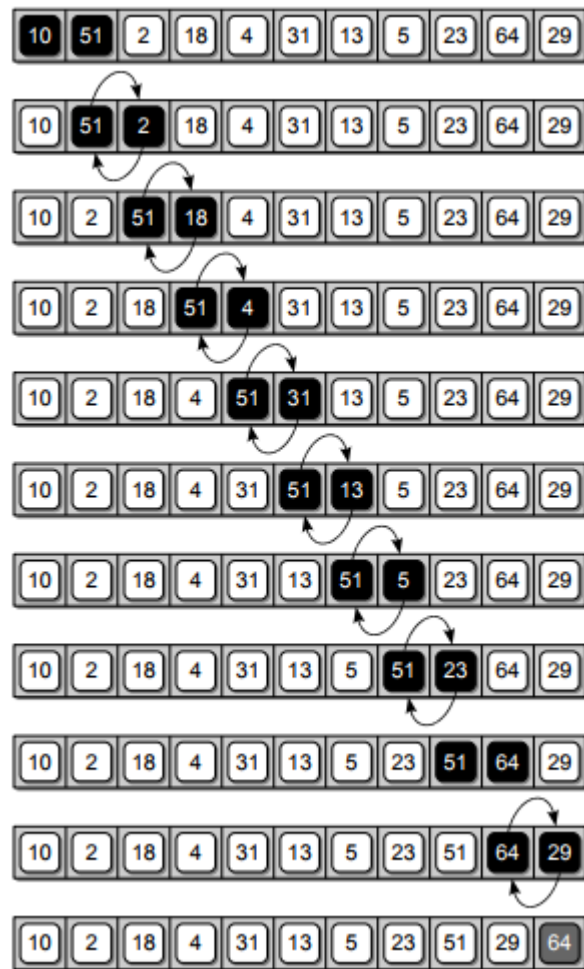
[End of if]

(B) Set $j = j+1$

[End of Inner Loop]

[End of Step 1 Outer Loop]

Step 4: Exit



Various Passes of Bubble Sort

Source Code:

Python program for implementation of Bubble Sort

```
def bubbleSort(arr):
```

```
    n = len(arr)
```

```
    # Traverse through all array elements
```

```
    for i in range(n):
```

```
        # Last i elements are already in place
```

```
        for j in range(0, n-i-1):
```

```
            # traverse the array from 0 to n-i-1
```

```
            # Swap if the element found is greater
```

```
            # than the next element
```

```
            if arr[j] > arr[j+1]:
```

```
arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
# Driver code to test above
```

```
arr = [64, 34, 25, 12, 22, 11, 90]
```

```
bubbleSort(arr)
```

```
print ("Sorted array is:")
```

```
for i in range(len(arr)):
```

```
    print ("%d" %arr[i])
```

Step-by-step example:

Let us take the array of numbers "5 1 4 2 8", and sort the array from lowest number to greatest number using bubble sort. In each step, elements written in **bold** are being compared. Three passes will be required.

First Pass:

(**5** **1** 4 2 8) (**1** **5** 4 2 8), Here, algorithm compares the first two elements, and swaps since $5 > 1$.

(**1** **5** **4** 2 8) (**1** **4** **5** 2 8), Swap since $5 > 4$

(**1** **4** **5** 2 8) (**1** **4** **2** **5** 8), Swap since $5 > 2$

(**1** **4** **2** **5** **8**)

(**1** **4** **2** **5** **8**), Now, since these elements are already in order ($8 > 5$), algorithm does not swap them.

Second Pass:

(**1** **4** **2** 5 8) (**1** **4** **2** 5 8)

(**1** **4** **2** 5 8) (**1** **2** **4** 5 8), Swap since $4 > 2$

(**1** **2** **4** 5 8) (**1** **2** **4** 5 8)

(**1** **2** **4** **5** **8**) (**1** **2** **4** **5** **8**)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one **whole** pass without **any** swap to know it is sorted.

Third Pass:

(**1** **2** **4** 5 8) (**1** **2** **4** 5 8)

(**1** **2** **4** 5 8) (**1** **2** **4** 5 8)

(**1** **2** **4** 5 8) (**1** **2** **4** 5 8)

(1 2 4 **5 8**) (1 2 4 **5 8**)

Time Complexity:

The efficiency of Bubble sort algorithm is independent of number of data items in the array and its initial arrangement. If an array containing n data items, then the outer loop executes n-1 times as the algorithm requires n-1 passes. In the first pass, the inner loop is executed n-1 times; in the second pass, n-2 times; in the third pass, n-3 times and so on. The total number of iterations resulting in a run time of $O(n^2)$.

Worst Case Performance	$O(n^2)$
Best Case Performance	$O(n^2)$
Average Case Performance	$O(n^2)$

Selection Sort:

Selection sort algorithm is one of the simplest sorting algorithm, which sorts the elements in an array by finding the minimum element in each pass from unsorted part and keeps it in the beginning. This sorting technique improves over bubble sort by making only one exchange in each pass. This sorting technique maintains two sub arrays, one sub array which is already sorted and the other one which is unsorted. In each iteration the minimum element (ascending order) is picked from unsorted array and moved to sorted sub array..

Selection Sort Algorithm:

```
For i = 1 to n-1 /*where n is the size of the array */
    set: min_index=1 /*here we took starting index as 1 */
    For j=i+1 to n
        If arr[j]<arr[min_index]
            set: min_index=j
        EndIf
    EndFor
    swap (arr[i] and swap[min_index])
EndFor
). Exit.
```

Source Code:

```
# Python program for implementation of Selection
# Sort
import sys
A = [64, 25, 12, 22, 11]

# Traverse through all array elements
for i in range(len(A)):
```

```

# Find the minimum element in remaining
# unsorted array
min_idx = i
for j in range(i+1, len(A)):
    if A[min_idx] > A[j]:
        min_idx = j

# Swap the found minimum element with
# the first element
A[i], A[min_idx] = A[min_idx], A[i]

```

```

# Driver code to test above
print ("Sorted array")
for i in range(len(A)):
    print("%d" %A[i])

```

Output:

Enter array size:6

Enter the elements:96 94 81 56 76 45

The elements after sorting are: 45 56 76 81 94 96

Step-by-step example:

Here is an example of this sort algorithm sorting five elements:

64 25 12 22 11

11 **25** **12** 22 64

11 **12** **25** 22 64

11 12 **22** **25** 64

11 12 22 25 64

Time Complexity:

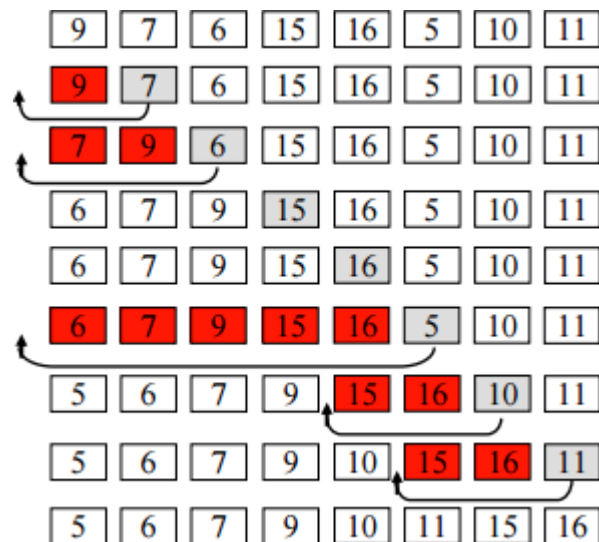
Selection sort is not difficult to analyze compared to other sorting algorithms since none of the loops depend on the data in the array. Selecting the lowest element requires scanning all n elements (this takes $n - 1$ comparisons) and then swapping it into the first position. Finding the next lowest element requires scanning the remaining $n - 1$ elements and so on, for $(n - 1) + (n - 2) + \dots + 2 + 1 = n(n - 1) / 2 \in O(n^2)$ comparisons. Each of these scans requires one swap for $n - 1$ elements (the final element is already in place).

Worst Case Performance	$O(n^2)$
Best Case Performance	$O(n^2)$

Insertion Sort:

An algorithm consider the elements one at a time, inserting each in its suitable place among those already considered (keeping them sorted). Insertion sort is an example of an **incremental** algorithm. It builds the sorted sequence one number at a time. This is a suitable sorting technique in playing card games. Insertion sort provides several advantages:

- Simple implementation
- Efficient for (quite) small data sets
- Adaptive (i.e., efficient) for data sets that are already substantially sorted: the time complexity is $O(n + d)$, where d is the number of inversions
- More efficient in practice than most other simple quadratic (i.e., $O(n^2)$) algorithms such as selection sort or bubble sort; the best case (nearly sorted input) is $O(n)$
- Stable; i.e., does not change the relative order of elements with equal keys
- In-place; i.e., only requires a constant amount $O(1)$ of additional memory space
- Online; i.e., can sort a list as it receives it

*Source Code:*

```
# Python program for implementation of Insertion Sort
```

```
# Function to do insertion sort
```

```
def insertionSort(arr):
```

```
    # Traverse through 1 to len(arr)
```

```
    for i in range(1, len(arr)):
```



```
key = arr[i]
```

```
# Move elements of arr[0..i-1], that are  
# greater than key, to one position ahead  
# of their current position
```

```
j = i-1
```

```
while j >= 0 and key < arr[j] :
```

```
    arr[j+1] = arr[j]
```

```
    j -= 1
```

```
arr[j+1] = key
```

```
# Driver code to test above
```

```
arr = [12, 11, 13, 5, 6]
```

```
insertionSort(arr)
```

```
print ("Sorted array is:")
```

```
for i in range(len(arr)):
```

```
    print ("%d" %arr[i])
```

Step-by-step example:

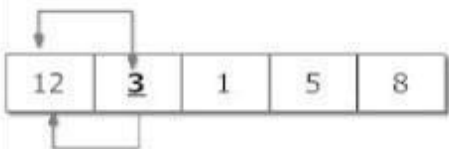
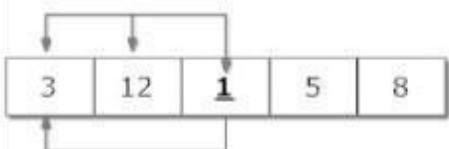
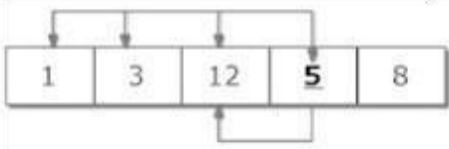
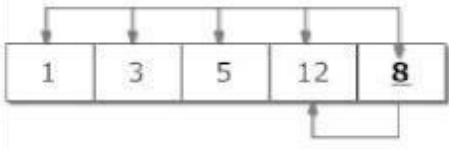
Step 1		Checking second element of array with element before it and inserting it in proper position. In this case, 3 is inserted in position of 12.
Step 2		Checking third element of array with elements before it and inserting it in proper position. In this case, 1 is inserted in position of 3.
Step 3		Checking fourth element of array with elements before it and inserting it in proper position. In this case, 5 is inserted in position of 12.
Step 4		Checking fifth element of array with elements before it and inserting it in proper position. In this case, 8 is inserted in position of 12.
		Sorted Array in Ascending Order

Figure: Sorting Array in Ascending Order Using Insertion Sort Algorithm

Suppose, you want to sort elements in ascending as in above figure. Then,

1. The second element of an array is compared with the elements that appear before it (only first element in this case). If the second element is smaller than first element, second element is inserted in the position of first element. After first step, first two elements of an array will be sorted.
2. The third element of an array is compared with the elements that appears before it (first and second element). If third element is smaller than first element, it is inserted in the position of first element. If third element is larger than first element but, smaller than second element, it is inserted in the position of second element. If third element is larger than both the elements, it is kept in the position as it is. After second step, first three elements of an array will be sorted.
3. Similarly, the fourth element of an array is compared with the elements that appear before it (first, second and third element) and the same procedure is applied and that element is inserted in the proper position. After third step, first four elements of an array will be sorted.

If there are n elements to be sorted. Then, this procedure is repeated $n-1$ times to get sorted list of array.

Time Complexity:

Worst Case Performance	$O(n^2)$
Best Case Performance(nearly)	$O(n)$
Average Case Performance	$O(n^2)$

Output:

Enter no of elements:5

Enter elements:1 65 0 32 66

Elements after sorting: 0 1 32 65 66

Quick Sort :

Quick sort is a divide and conquer algorithm. Quick sort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quick sort can then recursively sort the sub-lists.

The steps are:

1. Pick an element, called a **pivot**, from the list.
2. Reorder the list so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the **partition** operation.
3. Recursively apply the above steps to the sub-list of elements with smaller values and separately the sub-list of elements with greater values.

The base case of the recursion is lists of size zero or one, which never need to be sorted.

Quick sort, or **partition-exchange sort**, is a sorting algorithm developed by Tony Hoare that, on average, makes $O(n \log n)$ comparisons to sort n items. In the worst case, it makes $O(n^2)$ comparisons, though this behavior is rare. Quick sort is often faster in practice than other $O(n \log n)$ algorithms. It works by first of all by partitioning the array around a pivot value and then dealing with the 2 smaller partitions separately. Partitioning is the most complex part of quick sort. The simplest thing is to use the first value in the array, $a[l]$ (or $a[0]$ as $l = 0$ to begin with) as the pivot. After the partitioning, all values to the left of the pivot are $\leq$ pivot and all values to the right are $>$ pivot. The same procedure for the two remaining sub lists is repeated and so on recursively until we have the entire list sorted.

Advantages:

- One of the fastest algorithms on average.
- Does not need additional memory (the sorting takes place in the array - this is called **in-place** processing).

Disadvantages: The worst-case complexity is $O(N^2)$

Source Code:

Python program for implementation of Quicksort Sort

```
# This function takes last element as pivot, places
# the pivot element at its correct position in sorted
# array, and places all smaller (smaller than pivot)
# to left of pivot and all greater elements to right
# of pivot
```

```
def partition(arr,low,high):
```

```
    i = ( low-1 )      # index of smaller element
    pivot = arr[high]  # pivot
```

```
    for j in range(low , high):
```

```
        # If current element is smaller than or
        # equal to pivot
        if arr[j] <= pivot:
```

```
            # increment index of smaller element
            i = i+1
            arr[i],arr[j] = arr[j],arr[i]
```

```
    arr[i+1],arr[high] = arr[high],arr[i+1]
    return ( i+1 )
```

```
# The main function that implements QuickSort
# arr[] --> Array to be sorted,
# low --> Starting index,
# high --> Ending index
```

```

# Function to do Quick sort
def quickSort(arr,low,high):
    if low < high:

        # pi is partitioning index, arr[p] is now
        # at right place
        pi = partition(arr,low,high)

        # Separately sort elements before
        # partition and after partition
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)

# Driver code to test above
arr = [10, 7, 8, 9, 1, 5]
n = len(arr)
quickSort(arr,0,n-1)
print ("Sorted array is:")
for i in range(n):
    print ("%d" %arr[i])

```

Step-by-step example:

1	2	3	4	5	6	7	8	9	10	11	12	13	Remarks
38	08	16	06	79	57	24	56	02	58	04	70	45	
Pivot	08	16	06	Up	57	24	56	02	58	Dn	70	45	Swap up and down
Pivot	08	16	06	04	57	24	56	02	58	79	70	45	
Pivot	08	16	06	04	Up	24	56	Dn	58	79	70	45	Swap up and down
Pivot	08	16	06	04	02	24	56	57	58	79	70	45	
Pivot	08	16	06	04	02	Dn	Up	57	58	79	70	45	Swap pivot and down
24	08	16	06	04	02	38	56	57	58	79	70	45	
Pivot	08	16	06	04	Dn	Up	56	57	58	79	70	45	Swap pivot and down
(02	08	16	06	04	24)	38	(56	57	58	79	70	45)	
Pivot	08	16	06	04	Up								

dn

Pivot Up 06 Dn

Swap up and down

Pivot 04 06 16

Pivot 04 **Dn** **Up**

Swap pivot
and down

06 04 08

Pivot	Dn	Up
-------	----	----

**Swap pivot
and down**

04 06

(02 04 06 08 16 24 38) (56 57 58 79 70 45)

Swap up and down

Pivot	Up	58	79	70	Dn
--------------	-----------	----	----	----	-----------

Pivot 45 58 79 70 57

**Swap pivot
and down**

Pivot	Dn	Up	79	70	57
--------------	-----------	-----------	----	----	-----------

(45) 56 (58 79 70 57)

Swap up and down

Pivot	Up	70	Dn
-------	----	----	----

Pivot	57	70	79
--------------	-----------	----	-----------

Swap down and pivot

Pivot	Dn	Up	79
-------	----	----	----

Swap pivot
and down

Time Complexity:

Best Case
Performance(nearl
y)

Worst Case Performance

$$O(n^2)$$
 $O(n \log_2 n)$

Average Case
Performance

The array is sorted

$O(n \log_2 n)$

Merge Sort:

Merge sort is based on Divide and conquer method. It takes the list to be sorted and divide it in half to create two unsorted lists. The two unsorted lists are then sorted and merged to get a sorted list. The two unsorted lists are sorted by continually calling the merge-sort algorithm; we eventually get a list of size 1 which is already sorted. The two lists of size 1 are then merged.

Merge Sort Procedure:

This is a divide and conquer algorithm.

This works as follows :

1. Divide the input which we have to sort into two parts in the middle. Call it the left part and right part.
2. Sort each of them separately. Note that here sort does not mean to sort it using some other method. We use the same function recursively.
3. Then merge the two sorted parts.

Input the total number of elements that are there in an array (number_of_elements). Input the array (array[number_of_elements]). Then call the function MergeSort() to sort the input array. MergeSort() function sorts the array in the range [left,right] i.e. from index left to index right inclusive. Merge() function merges the two sorted parts. Sorted parts will be from [left, mid] and [mid+1, right]. After merging output the sorted array.

MergeSort() function:

It takes the array, left-most and right-most index of the array to be sorted as arguments. Middle index (mid) of the array is calculated as $(\text{left} + \text{right})/2$. Check if $(\text{left} < \text{right})$ cause we have to sort only when $\text{left} < \text{right}$ because when $\text{left} = \text{right}$ it is anyhow sorted. Sort the left part by calling MergeSort() function again over the left part MergeSort(array,left,mid) and the right part by recursive call of MergeSort function as MergeSort(array,mid + 1, right). Lastly merge the two arrays using the Merge function.

Merge() function:

It takes the array, left-most , middle and right-most index of the array to be merged as arguments.

Finally copy back the sorted array to the original array.

Source Code:

Recursive Python Program for merge sort

```
def merge(left, right):
    if not len(left) or not len(right):
        return left or right

    result = []
    i, j = 0, 0
    while (len(result) < len(left) + len(right)):
        if left[i] < right[j]:
            result.append(left[i])
```

```

        i+= 1
    else:
        result.append(right[j])
        j+= 1
    if i == len(left) or j == len(right):
        result.extend(left[i:] or right[j:])
        break

    return result

def mergesort(list):
    if len(list) < 2:
        return list

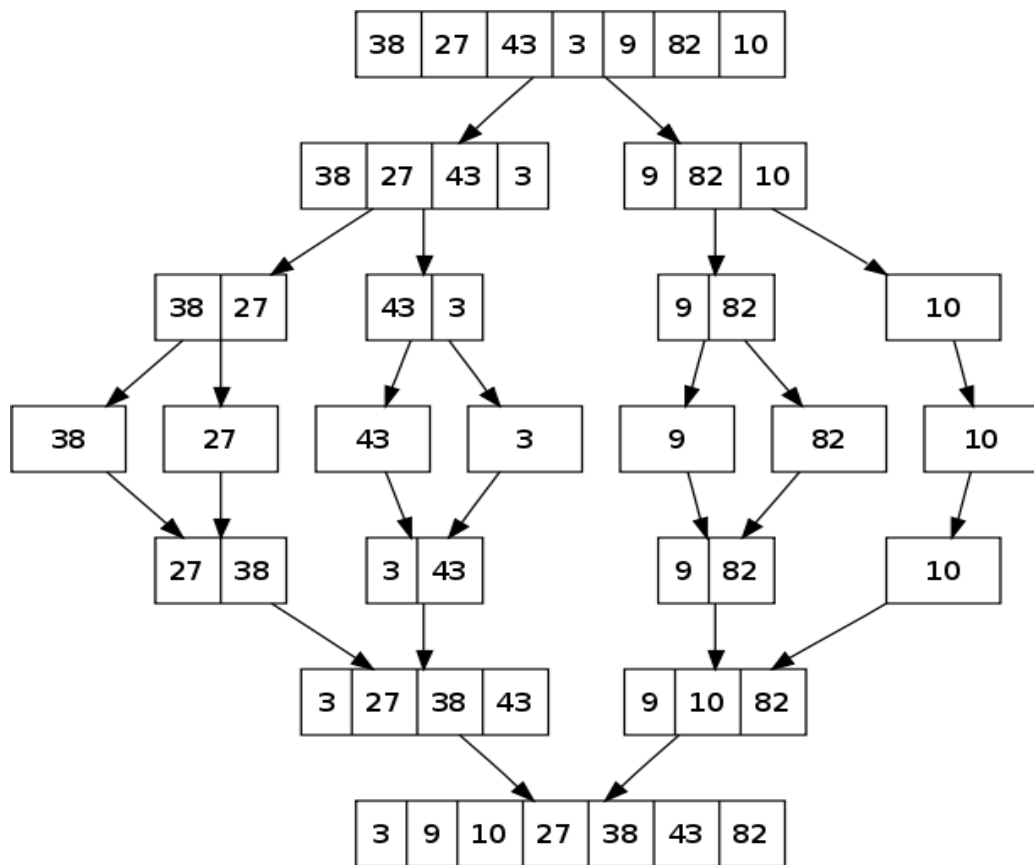
    middle = int(len(list)/2)
    left = mergesort(list[:middle])
    right = mergesort(list[middle:])

    return merge(left, right)

seq = [12, 11, 13, 5, 6, 7]
print("Given array is")
print(seq);
print("\n")
print("Sorted array is")
print(mergesort(seq))

```

Step-by-step example:



Merge Sort Example

Time Complexity:

Worst Case Performance	$O(n \log_2 n)$
Best Case Performance(nearly)	$O(n \log_2 n)$
Average Case Performance	$O(n \log_2 n)$

Comparison of Sorting Algorithms:

Time Complexity comparison of Sorting Algorithms:

Algorithm	Data Structure	Time Complexity		
		Best	Average	Worst
Quicksort	Array	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$
Mergesort	Array	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Bubble Sort	Array	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	Array	$O(n)$	$O(n^2)$	$O(n^2)$
Select Sort	Array	$O(n^2)$	$O(n^2)$	$O(n^2)$

Space Complexity comparison of Sorting Algorithms:

Algorithm	Data Structure	Worst Case Auxiliary Space Complexity
Quicksort	Array	$O(n)$
Mergesort	Array	$O(n)$
Bubble Sort	Array	$O(1)$
Insertion Sort	Array	$O(1)$
Select Sort	Array	$O(1)$
Bucket Sort	Array	$O(nk)$